Dead Internet Detector: Participant-Level Authenticity Reading of Pasted Social Threads

Course: NLP Group Project (Option 1, Application Development)

Group: Apilash Balasingham, Luis Guareschi, Alejandro Gutierrez Werner

Artifact: Streamlit proof of concept, evaluation harness, and tiered gold set (`app.py`, `src/`, `data/eval/participants.jsonl`)

Abstract

We present the Dead Internet Detector, a research prototype that reads a pasted social media thread and estimates, for each participant, whether the writing reads as a genuine human, an automated bot, a human performing a robotic voice, or a bot performing a casual human voice. The tool returns a confidence score and quoted cues for every judgment, and it aggregates a thread-level Dead Internet Index. The system combines lightweight stylometric features with a locally hosted language model, and it degrades gracefully to a fully offline heuristic mode when no model is available. We evaluate on a custom set of 48 participant-level gold labels drawn from five tiers of difficulty. The hybrid configuration reaches 0.71 accuracy and 0.57 macro-F1, against 0.60 accuracy and 0.51 macro-F1 for the heuristic baseline. Performance is strong on disclosed bots and plainly human replies and weak on the two imitation categories, where the distinguishing signal is authorial intent that the text often does not carry. We treat this gap as a finding about the task itself, not a defect to be engineered away, and we frame all output as probabilistic impressions, never accusations.

---

1. Introduction

Online forums and comment sections are a primary venue for public conversation, and a recurring question shadows them: who, or what, is actually speaking. The "Dead Internet" narrative captures a growing unease that much of what looks like organic discussion is automated, templated, or assisted by large language models. The concern is partly cultural and partly practical. Moderators cannot manually audit every participant in every thread, and ordinary readers have only informal heuristics to fall back on, such as the sense that a particular reply "feels like ChatGPT."

The problem we target is narrow and direct: given only the pasted text of a thread, with no account history, follower graph, or platform-internal signal, can a tool give a reader a structured, transparent reading of each participant? This is a harder setting than most deployed bot detectors operate in, because those systems lean heavily on account-level metadata that a paste-only tool cannot see.

Our contribution is threefold. First, we build a working proof of concept that classifies each participant into a four-label taxonomy and explains itself with quoted evidence. Second, we

design a tiered evaluation methodology that acknowledges, rather than hides, that perfect ground truth is unavailable for arbitrary pasted text. Third, we document a failure taxonomy that locates the hardest cases where the underlying NLP problem is genuinely ill-posed. The intended use is research and demonstration, and we are explicit throughout that the tool produces statistical impressions, not verdicts.

2. Related work

This section expands the industry context recorded in our field review (`docs/FIELD_REVIEW.md`) with the relevant research lines.

Social bot detection. The first generation of work framed the problem as classifying automated accounts. Botometer, originally BotOrNot, popularized a public account-level bot score for Twitter using features such as posting rate, follower ratios, and network structure (Davis et al., 2016; Yang et al., 2020). Surveys of this period describe a decade of detector and adversary co-evolution and emphasize that the dominant framing was binary and operated at the profile level, not the comment level (Ferrara et al., 2016; Cresci, 2020). Simulation studies stress-test detectors against adaptive bots, which underlines that any published signal eventually gets gamed.

Machine-generated text detection. Large language models shifted the threat model from spam templates to fluent, on-topic replies. A parallel research line detects whether a passage was machine-generated. GLTR visualizes per-token model likelihoods to expose statistically "too predictable" text (Gehrmann et al., 2019). Classifier-based detectors, including RoBERTa models trained on generator output, were released alongside GPT-2 (Solaiman et al., 2019), and later commercial detectors followed. This line of work moved the judgment from the account level to the text level, pairing a human comment with a model-written comment on the same topic. Our gold set adopts that paired format in its self-constructed grid tier, which we label "GRiD-style" to mark the construction; it is not drawn from an external benchmark of that name. A consistent caveat in this literature is that detection accuracy degrades as generators improve and as text gets shorter.

Human baseline studies. Experiments that ask people to distinguish human from machine text generally find that they are reliable on obvious cases and approach chance on hard ones. We treat this pattern as a working assumption rather than a cited result, and it informs two design choices: it sets a realistic ceiling for an automated tool, and it lets us read annotator disagreement as signal about case difficulty rather than as noise.

Stylometry and authorship attribution. Classical authorship attribution infers writing-style fingerprints from features such as function-word frequencies, lexical richness, and punctuation habits (Stamatatos, 2009). Our heuristic feature set is a small, interpretable stylometric layer in this tradition, chosen for transparency and offline reproducibility, with no claim to state-of-the-art accuracy.

The gap we target. Deployed and research systems mostly output a binary judgment (human vs. bot, or human-written vs. AI-generated) and frequently rely on account context. Few expose

imitation categories such as "human performing bot voice," and few operate purely on pasted text. Our prototype occupies that gap as a transparent, participant-level reader.

3. System design

3.1 Architecture

The pipeline has four stages, each isolated in its own module:

```
pasted text / Reddit URL
        │
        ▼
  parse_thread.py      → {username: [comments]}        (+ timestamps if from
Reddit)
        │
        ▼
  features.py          → per-user stylometric signals + heuristic label
        │
        ▼
  classify.py          → Ollama LLM reading (hybrid) or heuristic fallback
        │
        ▼
  aggregate.py         → Dead Internet Index + label counts
        │
        ▼
  app.py / pages/      → annotated thread, per-user cards, export, dashboard
```

The parser (`src/parse_thread.py`) accepts several common paste formats, including `u/name:` , `/u/name –` , `Author: name` , `[name]:` , and Reddit score lines such as `name 42 points` . Lines without a recognizable prefix attach to the most recent speaker, and a fallback splits unstructured paste into anonymous `user_N` blocks. This tolerance matters because real pasted threads are messy.

3.2 Label taxonomy

The taxonomy has four classes, defined in `docs/LABELING_GUIDELINES.md` and enforced by the schema in `src/schemas.py` :

Label	Definition	Illustrative cue
`human`	Genuine human voice: specific, reactive, inconsistent across comments	"nah the bus was 40 min late"
`bot`	Automated or LLM output without casual mimicry	"I am a bot. This action was performed automatically."
`human_imitating_bot`	Human performing a robotic voice as parody	"Greetings, fellow humans." / copypasta
`bot_imitating_human`	Bot performing casual human voice to drive engagement	"Great question! Absolutely agree, though it might be worth..."

The two imitation classes are the conceptual core of the project and, as the evaluation shows, the hardest to separate, because the difference rests on intent, which surface form does not always encode.

3.3 Feature layer

The heuristic layer (`src/features.py`) computes interpretable per-user signals: comment count and average length, type-token ratio (lexical diversity), emoji density, counts of known bot or LLM template phrases, hedging frequency, exclamation rate, a list-formatting score, an all-caps rate, and a generic-opener score that fires on engagement-bait phrasing such as "Great question" or "Totally agree." When timestamps are available from a Reddit load, the layer also computes `reply_interval_cv`, the coefficient of variation of gaps between a user's comments, on the intuition that mechanically uniform timing is a weak bot signal.

A transparent scoring function maps these signals to a label and a confidence. Each signal contributes additively to one of three non-human scores, the residual mass goes to `human`, and the highest score wins. Confidence is a bounded transform of the winning score. This component is deliberately simple so that every heuristic decision can be explained and reproduced offline.

3.4 Prompt and language-model reading

In hybrid mode the system passes the full thread, the target participant's comments, and the heuristic feature summary to a locally hosted model through Ollama (default `llama3.2:3b`, temperature 0.2). The system prompt (`prompts/classify_user.txt`) defines the four labels, gives a focused contrast between `human` and `bot_imitating_human`, and imposes two discipline rules: cite phrases only from the user's own comments, and keep confidence below 50 when evidence is thin. The model must return a strict JSON object, which is parsed defensively and validated against the schema. If the model returns an unknown label, the system falls back to the heuristic label rather than failing.

3.5 Modes and reliability

Three modes support both deployment and ablation. `features_only` runs entirely offline and is the reproducible baseline. `llm_only` uses the model without the heuristic prior. `hybrid` gives the model the heuristic summary as context. The model call runs under a 40-second timeout in a worker thread, and any failure in `llm_only` or `hybrid` falls back to the heuristic classifier, so the tool always returns a result. Two post-processing rules curb overconfidence: a single short comment caps confidence (to 55 in the offline path, 60 in the hybrid path), and any confidence below 40 sets a `low_evidence` flag surfaced in the interface.

3.6 Aggregation

The Dead Internet Index (`src/aggregate.py`) summarizes a thread in one number. Each non-human participant contributes its confidence divided by 100, humans contribute zero, and the sum is averaged over participants and scaled to 0–100. The index is therefore a confidence-weighted share of non-human participants, intended as a quick readout rather than a precise measurement.

4. Evaluation methodology

4.1 Why ground truth is hard

For arbitrary pasted text there is no oracle. The author's intent, the account's automation status, and the cultural register of a joke are frequently unrecoverable from the words alone. We therefore do not claim ground truth in general. Instead we construct a gold set whose labels are defensible by construction, and we separate confident cases from deliberately hard ones.

4.2 Gold set construction

The gold set (`data/eval/participants.jsonl`) holds 48 participant-level labels across five tiers:

Tier	Count	Basis for the label
`synthetic`	16	Team-authored role-play threads covering all four classes; labels high-confidence by design
`grid`	10	Team-constructed "GRiD-style" pairs: a casual human comment and a GPT comment on the same topic
`edge`	10	Hard cases: very short comments, sarcasm, copypasta, multilingual text, mod bots, coordinated phrasing
`expert`	9	De-identified excerpts with majority-vote labels; split votes flagged
`disclosed`	3	Bots that explicitly disclose automation, alongside human replies

The label distribution is 18 `human`, 15 `bot`, 9 `bot_imitating_human`, and 6 `human_imitating_bot`, so the two imitation classes are the minority and the most consequential for macro-averaged metrics. Two expert-tier rows carry an `annotator_disagreement` flag, recording cases where annotators split.

4.3 Metrics

We report accuracy and macro-F1 over the four classes. Macro-F1 weights each class equally and therefore penalizes failure on the minority imitation classes, which is the property we care about most. We also report a binary collapse (human vs. non-human), which isolates the simpler question of whether a participant is a genuine human at all. For calibration we compare mean confidence on correct and incorrect predictions and count high-confidence errors, defined as wrong predictions with confidence at or above 70.

4.4 Ablation and reproducibility

The headline ablation contrasts the fully offline `features_only` baseline with `hybrid`; the third mode, `llm_only`, exists for diagnostics and is not part of the reported comparison. The harness (`src/eval_runner.py`) always runs `features_only` then `hybrid` on the full set, writes metrics to `results/eval_run.json`, and saves a confusion matrix per mode. The full run is reproduced with `python -m src.eval_runner`, which prompts for an Ollama model and shows a progress bar, or through the Streamlit Evaluation dashboard via "Run full suite." Precomputed metrics ship in the repository so the dashboard can display results without a local model through "View

precomputed results." The harness also supports per-tier filtering (`run_eval(..., tier_filter=...)`), and per-tier confusion matrices are saved to `results/`.

5. Results

5.1 Headline metrics

Metric	features_only	hybrid
Accuracy	0.604	0.708
Macro-F1 (4 classes)	0.506	0.571
Binary macro-F1 (human vs. non-human)	0.708	0.750
Mean confidence when correct	53.6	57.2
Mean confidence when wrong	54.2	52.5
High-confidence errors (conf ≥ 70)	0	0

Adding the language model improves accuracy by about 10 points and macro-F1 by about 6 points over the offline baseline. The binary task is easier than the four-way task in both configurations, which is the expected pattern given that the imitation distinctions are the hard part. Neither configuration produces a high-confidence error on this set, which suggests the confidence caps and the prompt's thin-evidence rule are doing useful work. With 48 examples and a single run per mode, these are point estimates only: they support a directional reading of which method is stronger and where, and we make no claim of statistical significance.

The calibration figures deserve a caveat. For `features_only`, mean confidence is essentially identical on correct and incorrect predictions (53.6 vs. 54.2), so the heuristic confidence carries little discriminative information. The hybrid model separates the two slightly in the right direction (57.2 vs. 52.5), but the margin is small, and confidence should be read as a coarse signal rather than a probability.

5.2 Per-class breakdown

Per-class F1 from `results/eval_run.json`:

Class	Support	features_only F1	hybrid F1
`bot`	15	0.56	0.87
`human`	18	0.72	0.75
`human_imitating_bot`	6	0.55	0.67
`bot_imitating_human`	9	0.20	0.00

The hybrid model is strong on `bot` (0.87 F1, with both precision and recall at 0.87) and reliable on `human` (0.75 F1 at perfect recall, meaning every true human is caught at the cost of some precision). The decisive weakness is `bot_imitating_human`, where hybrid scores 0.00: the model

recovers none of the nine cases. Human precision of 0.60 against a support of 18 implies roughly twelve non-humans were predicted `human`, so much of this error is absorbed into the `human` column. The heuristic baseline does marginally better on `bot_imitating_human` (0.20 F1) because its generic-opener rule occasionally catches engagement-bait phrasing, but neither configuration handles this class acceptably. The minority class `human_imitating_bot` improves under hybrid (0.55 to 0.67), helped by the thread-level context the model receives.

The confusion matrices in `results/confusion_matrix_features_only.png` and `results/confusion_matrix_hybrid.png` show this error visually, with the `bot_imitating_human` row emptying into the `human` column.

5.3 Reading of the results

The pattern is coherent. Disclosed and template-like bots are easy because they carry explicit surface tells. Plain humans are easy because the system defaults toward `human` when no non-human signal fires, which inflates human recall at some cost to precision. The errors cluster on the imitation classes, where a bot written to sound like a relaxed human leaves few stylometric fingerprints for a small local model to grip. Section 6 examines why this is a property of the task and not only of the model.

6. Failure taxonomy

The full catalog is in `docs/FAILURE_ANALYSIS.md`. We highlight the three most instructive cases.

Intent collapse between the two imitation classes. A human posting copypasta and a bot posting engagement bait both use a performative register. Separating `human_imitating_bot` from `bot_imitating_human`, and separating either from the nearby honest class, requires knowing why the text was written. That information is frequently absent from the text, so this confusion is the report's central limitation and not a tuning issue. The 0.00 F1 on `bot_imitating_human` is the quantitative shadow of this case.

Casual bots misread as human. GRiD-style items show that a model writing in a plain, informal tone ("nah the bus was 40 min late" in register) is harder to flag than a formally written bot. Short, casual machine text dominates the heuristic signals toward `human`, which is exactly the direction of the observed `bot_imitating_human` errors.

Language-model rationalization. In hybrid mode the model sometimes returns a plausible explanation for a wrong label, citing cues that read convincingly but do not establish automation. We mitigate this by requiring cues drawn only from the user's own text and by capping confidence on thin evidence, but post-hoc rationalization remains a known risk of using a generative model as a classifier, and it argues for treating the reasoning field as a prompt for human judgment and not as proof.

Additional documented cases include overconfident guesses on single-word comments (mitigated by the confidence cap), degraded performance on code-switched and non-English text given the English-centric features, and coordinated human pile-ons that resemble bot clusters.

7. Ethics and limitations

Probabilistic output and harassment risk. The tool outputs statistical impressions. A wrong `bot` label attached to a real person could fuel harassment, so the interface frames results as vibes rather than verdicts and the documentation discourages punitive use. We do not support automated moderation or account targeting in the prototype.

Paste-only blindness. The system sees only thread text. It has no access to account age, posting history, device signals, or network structure, which are the signals deployed systems rely on most. This is a deliberate scope choice that also bounds achievable accuracy.

English-centric features. The stylometric layer assumes English prose. Multilingual and code-switched threads degrade, and this population is underrepresented in the gold set.

Generative rationalization. As noted in the failure taxonomy, a language model can produce fluent but ungrounded justifications. Confidence scores are coarse and only weakly calibrated, and they should not be read as probabilities.

Small and partly synthetic evaluation. Beyond the significance caveat noted in §5.1, the gold set skews easier than open-world paste: the synthetic tier is clean by construction, and a single mislabeled item visibly moves macro-F1 given the class imbalance. The numbers should be read as evidence about this set, not as a deployable accuracy guarantee.

8. Future work

Several directions follow directly from the limitations. Modeling the reply tree, so that a participant is read in the context of who they answer, would supply the relational signal the current per-user view discards. Per-language feature normalization, or a multilingual model, would address the English-centric weakness. A larger and more balanced gold set, especially more real-world `bot_imitating_human` examples, would make macro-F1 more stable and let us study the hardest class properly. A formal human-baseline study, with annotators labeling a held-out slice under the same paste-only constraint, would establish the realistic ceiling and let us report tool performance relative to people instead of in absolute terms. Finally, confidence calibration, for example a post-hoc reliability adjustment fitted on held-out data, would make the confidence field trustworthy enough to drive interface behavior such as abstention.

9. Use of AI tools

We used AI coding assistants (Cursor with Claude) for repository scaffolding, boilerplate for the Streamlit application, the regex parser and pydantic schemas, the evaluation runner skeleton, and first drafts of documentation. The substantive decisions remained team-owned: the four-label taxonomy and its definitions, every gold label in `data/eval/participants.jsonl`, the authorship of the synthetic role-play threads, the expert-tier labeling and disagreement flags, the final prompt wording, the metric choices and thresholds, and the interpretation of failure modes. The decision to cap confidence on short comments is a transparent rule visible in the code. Team members can explain the parser behavior, the label definitions, and the evaluation metrics, and the full pipeline is reproducible from `requirements.txt` and the commands documented above.

References

Cresci, S. (2020). A decade of social bot detection. *Communications of the ACM*, 63(10), 72–83.

Davis, C. A., Varol, O., Ferrara, E., Flammini, A., & Menczer, F. (2016). BotOrNot: A system to evaluate social bots. *Proceedings of the 25th International Conference Companion on World Wide Web (WWW)*, 273–274.

Ferrara, E., Varol, O., Davis, C., Menczer, F., & Flammini, A. (2016). The rise of social bots. *Communications of the ACM*, 59(7), 96–104.

Gehrmann, S., Strobelt, H., & Rush, A. M. (2019). GLTR: Statistical detection and visualization of generated text. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 111–116.

Solaiman, I., Brundage, M., Clark, J., et al. (2019). Release strategies and the social impacts of language models. *arXiv:1908.09203*.

Stamatatos, E. (2009). A survey of modern authorship attribution methods. *Journal of the American Society for Information Science and Technology*, 60(3), 538–556.

Yang, K.-C., Varol, O., Hui, P.-M., & Menczer, F. (2020). Scalable and generalizable social bot detection through data selection. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(01), 1096–1103.